The Custody–Privacy Gap: What a Ten-Dollar Hardware Wallet Reveals About Self-Custody on Public Ledgers

Arnav Panjla[1] and Sahitya Shankar[1]
[1]Blockchain Society, IIT Delhi
arnavpanjla@gmail.com, yraretil@gmail.com

Abstract. Every self-custody hardware wallet is sold on one promise: your private key never leaves the device. Noir Wallet, an open-source Ethereum signer built on a $3–5 STM32F401CCU6 microcontroller (WeAct BlackPill board) and a $1–$2 ATECC608A secure element, is used here as a working case study to test a second, quieter promise that hardware wallets do not make but are often assumed to deliver: privacy. We trace the signing pipeline of this ten-dollar device stage by stage — secure element, firmware, USB HID transport, host software, broadcast node, public ledger — and show that hardware-anchored key custody leaves every one of these stages except the first fully observable. Address reuse, transaction-graph clustering, and host-side metadata survive perfect key custody untouched. We use a real firmware bug found on this device, a secure element whose random number generator produces a fixed diagnostic pattern until its configuration zone is permanently locked, as a concrete illustration of what a verifiable hardware security claim actually looks like, as opposed to a marketing one. We then map where privacy-enhancing techniques such as stealth addresses, zero-knowledge proofs of authorization, and selectively disclosable credentials would need to sit on top of this hardware to close the gap, and argue, in line with Ledger's own hardware-anchored trust model, that the cost of that root of trust should not be the reason self-custody privacy stays a privilege of a few hundred dollars.
Keywords: self-custody, secure elements, hardware wallets, zero-knowledge proofs, transaction privacy, chain surveillance, ATECC608A

1. Introduction

Bitcoin's original design already contained the tension this paper is about: ownership is pseudonymous, but the flow of funds is permanently and globally visible [1]. Ethereum inherited the same property. A hardware wallet solves a narrow, well-defined problem inside that system: it keeps the signing key away from a general-purpose computer that might be compromised. It says nothing about what happens to the transaction after it is signed.

This distinction gets collapsed in practice. Marketing copy, and sometimes users themselves, treat "self-custody" and "private" as near-synonyms. They are not. A hardware-signed transaction from a reused address is exactly as linkable, exactly as clusterable, and exactly as visible to chain-surveillance firms as one signed by a browser extension. The hardware only protects the key; it does not touch what the key signs or where the signed artifact travels afterward.

We investigate this gap using Noir Wallet, an open-source Ethereum hardware wallet built on an STM32F4 (WeAct BlackPill, ARM Cortex-M4F) microcontroller for roughly $10 in components, as a concrete substrate rather than a hypothetical one. Using its actual firmware, its actual secure element, and a real hardware bug discovered while building it, we ask three questions: (1) at which stage of a real hardware-wallet signing pipeline does privacy actually leak, (2) what can a secure element verifiably guarantee versus merely claim, and (3) does the cost of a hardware root of trust have to be the reason privacy-preserving self-custody remains inaccessible. Section 6 develops the cost point. Section 5 compares this prototype against published Ledger and Trezor architectures on two physical attacks the $10 build does not close.

2. Background

2.1 Pseudonymity is not privacy

Meiklejohn et al. showed that two simple heuristics, transactions sharing multiple input addresses controlled by one signer, and one-time change-address detection, are enough to collapse millions of Bitcoin addresses into a much smaller number of real-world clusters [1]. The same class of heuristic-based clustering generalizes to account-based chains like Ethereum through address-reuse and interaction-graph analysis. None of this requires breaking any cryptography; it only requires that the chain be public, which it is by design.

2.2 Privacy-enhancing primitives that exist today

Three lines of work are directly relevant to where a hardware signer sits in a privacy-preserving stack:

- **Zero-knowledge transaction privacy.** Zerocash demonstrated that zk-SNARKs can hide a transaction's origin, destination, and amount while still

letting the network verify that no value was created or destroyed [2]. The technique that matters for this paper is not the currency it produced, but the pattern: a proof of a valid state transition that reveals nothing about the transition itself.

- **Stealth addresses.** Ethereum's ERC-5564 standardizes a way for a sender to derive a one-time address for a recipient non-interactively, so that on-chain observers cannot link a payment to the recipient's known public identity [3]. This is a comparatively cheap privacy gain: it needs elliptic-curve key derivation, not a full proving system.
- **Selective disclosure.** The W3C Verifiable Credentials model, combined with SD-JWT-style selective disclosure, lets a holder reveal only the specific claims a verifier needs (e.g., "over 18") without exposing the full underlying credential [4]. This is the closest existing primitive to what a compliance-aware, privacy-preserving wallet would need to prove authorization without exposing identity.

2.3 The regulatory squeeze

Privacy tooling does not exist in a vacuum. The U.S. Treasury's 2022 sanctioning of the Tornado Cash mixer, later partly reversed by a 2024 appellate ruling that immutable smart contracts are not "property" subject to sanction, is the clearest example of privacy-preserving infrastructure colliding directly with financial regulation [6]. Any privacy design proposed on top of a hardware wallet has to be argued for, and defended, with this reality in view; it is one reason this paper treats selective disclosure (which is compliance-compatible by construction) as a more immediately practical target than full transaction shielding.

3. Methodology

This is a design-analysis paper, not a controlled lab experiment. The method is a structured architectural review of Noir Wallet's signing pipeline, firmware, and secure-element behavior, plus a comparison to published commercial architectures, in four steps:

1. **Decompose** the signing pipeline into discrete stages, from key generation inside the secure element to broadcast on the public chain.
2. **Classify** each stage's exposure: what an on-path or on-chain observer can see at that stage, independent of whether the key itself is ever compromised.
3. **Measure** the unlocked ATECC608A `Random` output as a Shannon-entropy object (Section 4.3), using the documented diagnostic stream rather than a claimed post-lock capture. Configuration lock is one-way; this build has not been locked.
4. **Compare** the same pipeline against published Ledger and Trezor hardware designs (Section 5), then map remaining leaks to protocol-layer mitigations (Section 7).

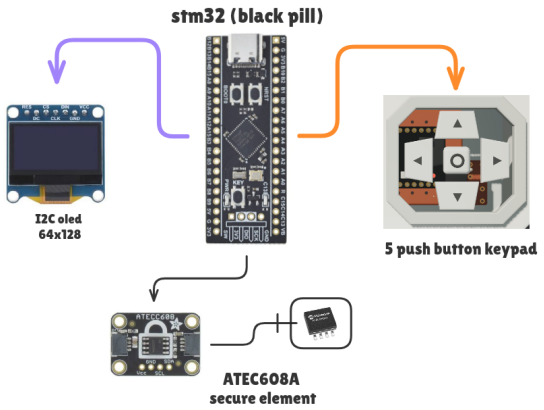

Figure 1: Noir Wallet hardware layout: STM32F401CCU6 MCU (BlackPill), SSD1306 OLED, ATECC608A secure element, and push-button input, all on a single I^2C/GPIO bus.

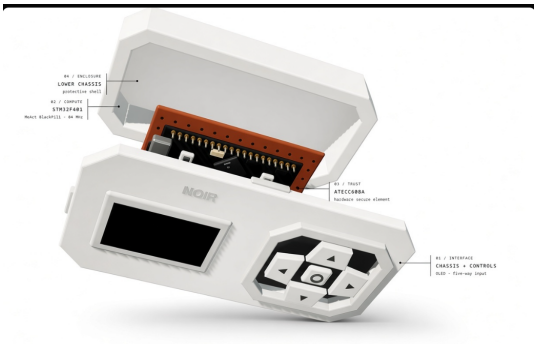

Figure 2: Enclosure CAD model.

This mirrors standard threat-modeling practice (enumerate assets, enumerate the attack surface at each trust boundary) applied specifically to the custody/privacy distinction rather than to key-extraction attacks, which is the threat model hardware wallets are already designed and marketed against.

4. Case Study: Noir Wallet

4.1 Architecture

Noir Wallet signs Ethereum transactions offline on an STM32F401CCU6 microcontroller (WeAct BlackPill board, ARM Cortex-M4F at 84 MHz, 64 KB SRAM, 256 KB flash), displays transaction details on a 128×64 OLED for human verification, requires a 4-digit PIN before any signing operation, and communicates with the host over USB HID, the same transport MetaMask and other browser wallets already speak. An ATECC608A secure element is wired onto the same I^2C bus as the display and used for key storage and randomness. Figure 1 shows the physical layout.

4.2 What the secure element actually guarantees

The ATECC608A is not a generic EEPROM; it is a cryptographic co-processor with hardware-enforced

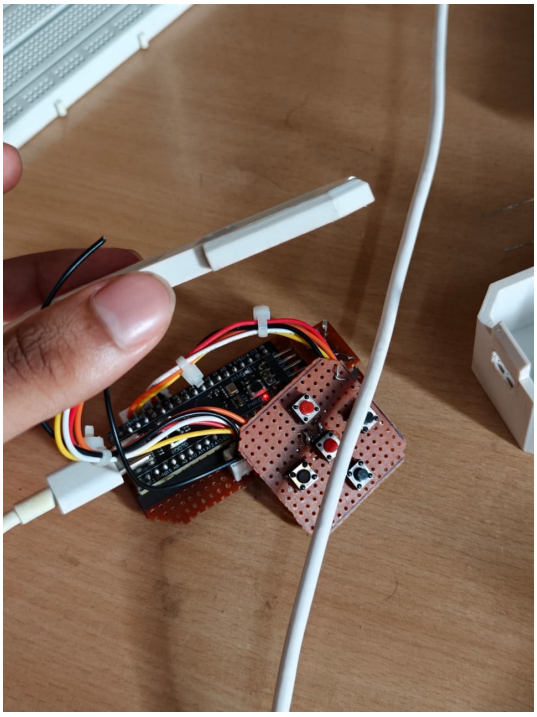

Figure 3: Assembled internals: STM32F401CCU6 board wired to the button matrix and OLED header before enclosure. Photograph of the physical build described in Section 4.1.

primitives [5]:

- **Non-exportable key storage** across 16 slots: a private key generated inside the chip can sign and derive, but the raw key material is never returned over the bus.
- **Hardware ECDSA/ECDH** on NIST P-256, so signing happens inside the secure boundary rather than in MCU RAM, which is a far larger and more frequently patched attack surface.
- **Hardware SHA-256/HMAC**, usable for firmware or command authentication independent of the MCU's own (much weaker) software stack.
- **An on-chip TRNG** specified against NIST SP 800-90A/B/C, with runtime health testing of the entropy source.
- **Monotonic counters and a one-way configuration lock**, which is the mechanism Section 4.3 turns into a worked example.

The important property for this paper is that every one of these is independently testable. A vendor claim like "bank-grade security" cannot be falsified by an outsider; a claim like "the private key never appears outside the secure element" can be checked by instrumenting the I^2C bus and confirming the key never transits it. This distinction, verifiable versus asserted, is what Section 8 returns to under the Ledger Lens. Section 5 is the complementary point: the same bus test also shows what the chip does *not* hide.

Figure 4: The open enclosure with OLED powered on, showing the physical arrangement referenced throughout Section 4.

4.3 A worked example: the TRNG that wasn't random

While bringing up this exact chip, we found that on an unconfigured, unlocked ATECC608A, the `Random` command (opcode `0x1B`) does not return entropy at all: it returns a fixed diagnostic pattern (`FF FF 00 00...`), by design, to make bring-up testing deterministic. Microchip confirms this behavior applies until the configuration zone is permanently locked. On this unit it produced a fixed 12-word BIP-39 mnemonic every time, since the "random" seed was not random. The fix is a single one-way command, `Lock` (`0x17`, mode `0x80`) on the configuration zone, after which the same command returns genuine entropy.

This is offered deliberately as a small, unglamorous bug rather than a success story. It illustrates exactly what "hardware-anchored trust" should mean in a paper rather than a datasheet: a property that can fail silently, that failed in a specific, reproducible, and now-documented way on this device, and that has a specific, verifiable fix. The current shipped firmware still uses a demo-grade entropy backstop (an MCU cycle-counter XOR) rather than the locked TRNG, and this is stated plainly rather than papered over: it is not a NIST-certified source. A paper making unfalsifiable security claims about its own hardware would be exactly the kind of assertion this section argues against.

Making "random" measurable. The bug is also why RNG claims need a repeated-invocation metric. Treated as one 32-byte draw, the pattern `FF FF 00 00...` still contains two symbols and can look "mixed." Across calls it does not: every invocation returns the same string, so the Shannon entropy of any fixed byte offset is $H = -\sum_i p_i \log_2 p_i = 0$ bits. Concatenating the documented diagnostic stream into 10,000 bytes yields only the symbols `0x00` and `0xFF` at equal fre-

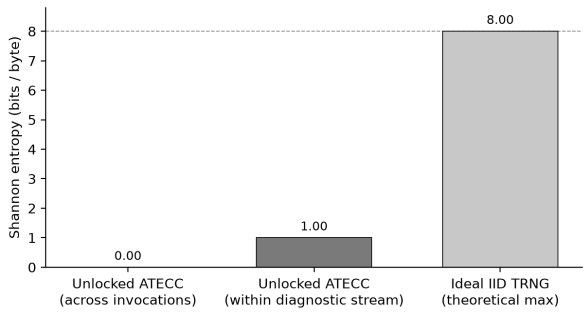

Figure 5: Shannon entropy of the unlocked ATECC608A diagnostic RNG versus the 8-bit/byte ceiling of an ideal IID byte source. Left and center bars are computed from the documented `FF FF 00 00` stream (10,000 concatenated bytes). The right bar is theoretical, not a locked-device measurement.

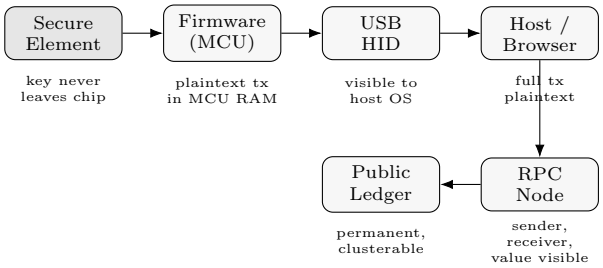

Figure 6: Exposure across the Noir Wallet signing pipeline. Only the leftmost stage is protected by the secure element; every stage after it is observable by an on-path party or, once broadcast, by anyone.

quency, so within-stream entropy is exactly 1.00 bit/byte. An ideal IID uniform byte source sits at 8 bits/byte. Figure 5 plots those three numbers. The right-hand bar is a theoretical ceiling, not a measured post-lock capture: lock is one-way, and this firmware has not taken that step. NIST SP 800-90B's Repetition Count Test and Adaptive Proportion Test are built to catch exactly this class of stuck source [7].

4.4 Where privacy actually leaks

Figure 6 traces the same pipeline for information exposure rather than key exposure. The secure element and PIN gate protect stage 1 completely: an attacker without the device and PIN cannot produce a valid signature. Every later stage is observable by design. USB HID payloads are visible to the host OS; the host (browser, MetaMask) sees the full transaction plaintext before broadcast; the broadcasting RPC node sees sender, receiver, and value; and the settled transaction is permanently public on-chain, where clustering heuristics such as Meiklejohn et al.'s apply regardless of how the signature was produced [1].

It is worth being precise about what the PIN and the offline-signing design *do* buy, since they are easy to conflate with privacy. The 4-digit PIN prevents an attacker who has physically stolen the device from signing on the owner's behalf; offline signing prevents a compromised host from silently exfiltrating the private key. Both are custody protections. Neither prevents the same compromised host from reading the transaction the device is about to sign, choosing to reuse an address, or logging metadata for later sale to a chain-analysis vendor, because the host is where the transaction is composed, and inspecting composed-but-unsigned data has always been outside a hardware wallet's threat model. This is not a flaw specific to Noir Wallet; it is a structural property of the "air-gapped signer" pattern used by essentially every hardware wallet on the market, and it is precisely the gap Sections 7 and 8 address.

Table 1: Custody model versus privacy guarantee.

Model	Key custody	Transaction privacy
Hot wallet	Weak (host RAM)	None
Hardware wallet	Strong (secure element)	None
Hardware wallet + stealth address	Strong	Recipient unlinkability only
Hardware wallet + ZK proof	Strong	Origin/amount hidden, needs off-chip proving

Table 1 summarizes this at the level of custody model rather than pipeline stage.

5. Hardware Threat Model and V1 Mitigations

Section 4 established that the ATECC608A can keep a private key off the wire. That is not the same as a trustworthy display, and it is not the same as a confidential bus. This section names two physical attacks that commercial designs spend their architecture on, and that Noir Wallet V0 fails. Neither attack extracts the key. Both break the user's ability to know what was signed, or who else saw the signing session.

5.1 Display substitution (compromised MCU)

On this board the STM32 owns the OLED and also owns the I^2C commands sent to the ATECC608A. A malicious or replaced firmware image can show "send 0.1 ETH to Alice" on the screen and submit a different digest to the secure element. The ATECC608A will sign whatever hash it is given. It has no path to the pixels. That is a failure of What-You-See-Is-What-You-Sign (WYSIWYS), not of key custody.

Ledger's published architecture inverts that trust: application logic and secrets run on an ST33-class secure element; an STM32 MCU is an untrusted proxy for USB and peripherals; display updates are issued by the SE over the SEPROXYHAL link rather than composed by the MCU as the source of truth [8]. Ledger's own write-up of a Nano X MCU debug issue states the same split: the MCU is not trusted, and it is not

supposed to be able to change what the secure display shows [9]. This paper does not claim that split is "mathematically impossible" to break. It claims the trust boundary is in a different place than on Noir Wallet, where the MCU is both the display driver and the SE client.

5.2 Plaintext I²C sniffing

The OLED (`0x3C`) and the ATECC608A (`0x60`) share one unencrypted I²C bus. A clip-on logic analyzer on SDA/SCL can record opcodes, slot indices, the digest sent for `Sign`, and the bytes written into the display buffer. The raw key still does not appear. Transaction metadata and screen contents do. The PIN on this firmware is checked on the MCU, so a tap of the ATECC lines does not automatically yield the PIN; a tap of the OLED lines does yield whatever the user was shown.

Trezor Safe 3 and Safe 5 add an Infineon OPTIGA Trust M (V3) beside the main MCU and use it for PIN-gated secrets, genuineness, and entropy [10]. That part supports a shielded I²C session (AES-128-CCM) between host MCU and secure element [11]. A bus tap of that channel sees ciphertext. It does not, by itself, encrypt the OLED. The relevant comparison for Noir Wallet is therefore narrower than a slogan: commercial designs encrypt or relocate the SE link; this prototype leaves that link in the clear, even though the ATECC608A datasheet already lists an encrypted I/O mode that this build has not turned on [5].

5.3 What V1 has to change

V0 protects remote key exfiltration. It does not protect local confidentiality or display integrity. The next revision does not need a closed ST33 to close the cheapest of these gaps: enable the ATECC608A I/O-protection secret so SE traffic is not plaintext; put the OLED on a separate bus or treat its framebuffer as attacker-visible; keep signing gated on a button event the user can correlate with the screen. A secure-element-owned display path, Ledger's published model, is the stronger WYSIWYS end-state and is also the expensive one. The open question for Section 8 is whether the encrypted-bus half of that stack can stay in the $10 class.

6. Cost as a Privacy Variable: A Ten-Dollar Root of Trust

Section 4.2 described a set of hardware guarantees, non-exportable keys, hardware ECDSA, an auditable TRNG, that are the same class of primitive found in commercial hardware wallets costing $60–$250. Noir Wallet builds those guarantees for roughly $10 in total: an STM32F401CCU6 board ($3–5), an SSD1306 OLED ($2–3), the ATECC608A secure element itself ($1–2), and a handful of push buttons and passives ($1).

This matters for the paper's argument for a reason that has nothing to do with the specific device: the marginal cost of hardware-anchored key custody is not the reason self-custody remains rare. A $1–2 secure element buys the same non-exportable-key and hardware-TRNG guarantees discussed in Section 4.2 regardless of whether it sits inside a $10 hobbyist board or a $150 commercial device. If, as Section 7 argues, future privacy tooling (stealth-address derivation, ZK-proof generation) needs to be anchored in hardware to be trustworthy, that anchor should be assumed to be cheap and widely reproducible, not treated as a premium feature bundled with brand and packaging. Section 5 is the limit of that claim: the $10 BOM buys key custody, not display integrity and not a confidential bus. This is an argued position, not a product claim. It says nothing about assurance level, certification, or tamper-resistance packaging, all of which commercial devices generally do better and which a $10 prototype explicitly does not attempt to match.

7. Where Privacy-Enhancing Technology Plugs In

Mapping Section 4.4's leak points onto Section 2's primitives gives a rough ordering by feasibility on constrained hardware like an STM32F401CCU6 (84 MHz Cortex-M4F, 64 KB SRAM, 256 KB flash):

Stealth addresses (cheapest). ERC-5564 only needs elliptic-curve scalar multiplication to derive a one-time address per payment [3]. This is well within reach of the same MCU that already does ECDSA signing (or can be offloaded to the ATECC608A's own ECC engine), and closes the recipient-linkability leak at the host and chain stages without touching the secure element's key-custody role at all.

Selective-disclosure authorization (moderate). A wallet could hold a signed credential (e.g., "this key is authorized to spend up to X") and disclose only the minimal claim needed at spend time, following the SD-JWT pattern [4]. The signing itself still happens on-chip; only the disclosure logic needs to move onto the MCU or a companion app.

Full ZK transaction shielding (hardest, currently infeasible on-device). Zerocash-class proving requires a large arithmetic circuit (millions of R1CS constraints) and large-integer modular arithmetic well beyond what 64 KB of SRAM can hold as working memory [2]; this is a memory and cycle-count ceiling, not one the MCU's hardware FPU changes, since SNARK proving is dominated by big-integer field arithmetic rather than floating-point operations. A realistic near-term design generates the proof off-device (e.g., on the paired phone or browser) while keeping only the authorizing signature on the secure element. This split is itself a privacy trade-off worth stating plainly rather than glossing over: the proof-generation host sees the private inputs, even if the chain never does.

8. Ledger Lens: Hardware Trust Is Necessary, Not Sufficient

Ledger's own position, self-custody, hardware-anchored trust, is a genuinely useful frame for this paper's central claim, and it is worth stating the critique in Ledger's own terms rather than around them. Hardware-anchored trust answers the question "can I prove this key was never exposed?" It does not answer, and was never designed to answer, "can I prove I sent value without revealing who I am or how much?" These are different guarantees, secured by different mechanisms, and Section 4 showed concretely (using a real chip, a real bug, and a real fix) that the first guarantee is verifiable in a way that matters: it can be tested independently of the vendor's word.

The honest version of the Ledger Lens for a privacy paper is this: any credible privacy-preserving wallet design still needs a hardware root of trust underneath it, because a signature or a ZK-proof authorization is only as trustworthy as the key that produced it, and software-only key storage has a long, well-documented history of exfiltration. Section 5 adds the physical half of that sentence. A $10 ATECC608A stops remote key theft. It does not stop a malicious STM32 from lying about the screen, and it does not stop a logic analyzer from reading the bus. Ledger's published answer to the first problem is an SE-owned display path [8, 9]. Trezor Safe 3/ 5's published answer to the second, on the SE link, is a shielded MCU–SE channel [10, 11]. Those are real engineering costs, not packaging. The unresolved question this paper wants on the table is narrower than "why does a hardware wallet cost $150?": does *true* self-custody — key isolation plus WYSIWYS plus a non-plaintext SE bus — have to stay at that price, or can an open $10 board take the encrypted-I^2C step without buying a closed display-controller SE? Hardware-anchored trust remains necessary. On this prototype it is also not sufficient, and the part that is cheap is only the key.

9. Limitations and Discussion

This is a design analysis of one open-source device, not an empirical study across hardware wallets, and it makes no claim that Noir Wallet's specific firmware is production-secure. The TRNG finding in Section 4.3 is an unlocked, demo-grade state, not a post-lock measurement. Figure 5's 8-bit bar is a ceiling, not a captured locked-TRNG dataset. The privacy-leak mapping in Section 4.4 and the two attacks in Section 5 are threat models, not lab traces from a logic analyzer, and they do not cover power or EM side channels against the ATECC608A. The cost argument in Section 6 is bill-of-materials only: no Common Criteria, no supply-chain assurance, no tamper-evident case.

Section 5 already carries the I^2C and display-integrity limits; they are not repeated here. A further limit is Section 7: feasibility only, no implemented stealth-address or proving pipeline.

10. Conclusion

Using an actual ten-dollar, open-source hardware wallet as the object of study rather than a hypothetical one, this paper traced where custody protection ends and privacy exposure begins: at the boundary of the secure element itself. A real firmware bug in that secure element's random number generator, scored as Shannon entropy of 0 bits across invocations and 1 bit/byte inside the diagnostic stream, served as a concrete demonstration of what a verifiable hardware-security claim looks like. Section 5 then named the two physical gaps this V0 does not close: a MCU that can lie about the screen, and a plaintext I^2C bus. Stealth addresses and selective disclosure are within reach of the same chip already doing the signing; full ZK shielding is not, on-device, today. Hardware-anchored trust remains the correct foundation. On this board it is cheap for the key, and still insufficient for display integrity and bus confidentiality.

Originality Statement

This paper is the original work of the listed author(s). All external claims are cited to their sources. AI tools were used as a working aid (research synthesis, drafting assistance); no section was generated end-to-end by an AI system without author review and rewriting. *[Signature / date placeholder — complete before submission.]*

References

[1] S. Meiklejohn, M. Pomarole, G. Jordan, K. Levchenko, D. McCoy, G. M. Voelker, S. Savage, "A Fistful of Bitcoins: Characterizing Payments Among Men with No Names," *Proc. Internet Measurement Conference (IMC)*, ACM, 2013, pp. 127–140.

[2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, M. Virza, "Zerocash: Decentralized Anonymous Payments from Bitcoin," *IEEE Symposium on Security and Privacy*, 2014.

[3] T. Wahrstätter, M. Solomon, B. DiFrancesco, V. Buterin, "ERC-5564: Stealth Addresses," Ethereum Improvement Proposals, 2022. `https://eips.ethereum.org/EIPS/eip-5564`

[4] World Wide Web Consortium, "Securing Verifiable Credentials using JOSE and COSE," W3C Recommendation, 2025. `https://www.w3.org/TR/vc-jose-cose/`

[5] Microchip Technology Inc., "ATECC608A: CryptoAuthentication Device Data Sheet," Doc. DS40001977A, 2019.

[6] U.S. Department of the Treasury, Office of Foreign Assets Control, "U.S. Treasury Sanctions Notorious Virtual Currency Mixer Tornado Cash," Press Release, Aug. 8, 2022. `https://home.treasury.gov/news/press-releases/jy0916`

[7] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, M. Boyle, "Recommendation for the

Entropy Sources Used for Random Bit Generation," NIST Special Publication 800-90B, Jan. 2018.

[8] Ledger, "Hardware architecture," Ledger Developer Portal. https://developers.ledger.com/docs/device-app/explanation/ledger-os/hardware-architecture

[9] Ledger, "Enhancing the Ledger Nano X's Security," Ledger blog, 2020. https://www.ledger.com/enhancing-the-ledger-nano-xs-security

[10] Trezor, "Secure Elements in Trezor Safe devices." https://trezor.io/learn/security-privacy/how-trezor-keeps-you-safe/secure-elements-in-trezor-safe-devices

[11] Infineon Technologies, "OPTIGA Trust M: Shielded Connection," Knowledge Base Article. https://community.infineon.com/t5/Knowledge-Base-Articles/OPTIGA-Trust-M-Shielded-connection/ta-p/354380